Sincronização e Deadlock

Fundamentos, Mecanismos e Estratégias de Tratamento em Sistemas Operacionais

Disciplina: Sistemas Operacionais
(Baseado em Silberschatz, Tanenbaum e Stallings)

Curso: Bacharelado em Ciência da Computação

Professor: Prof. Me. Juvenal Neto

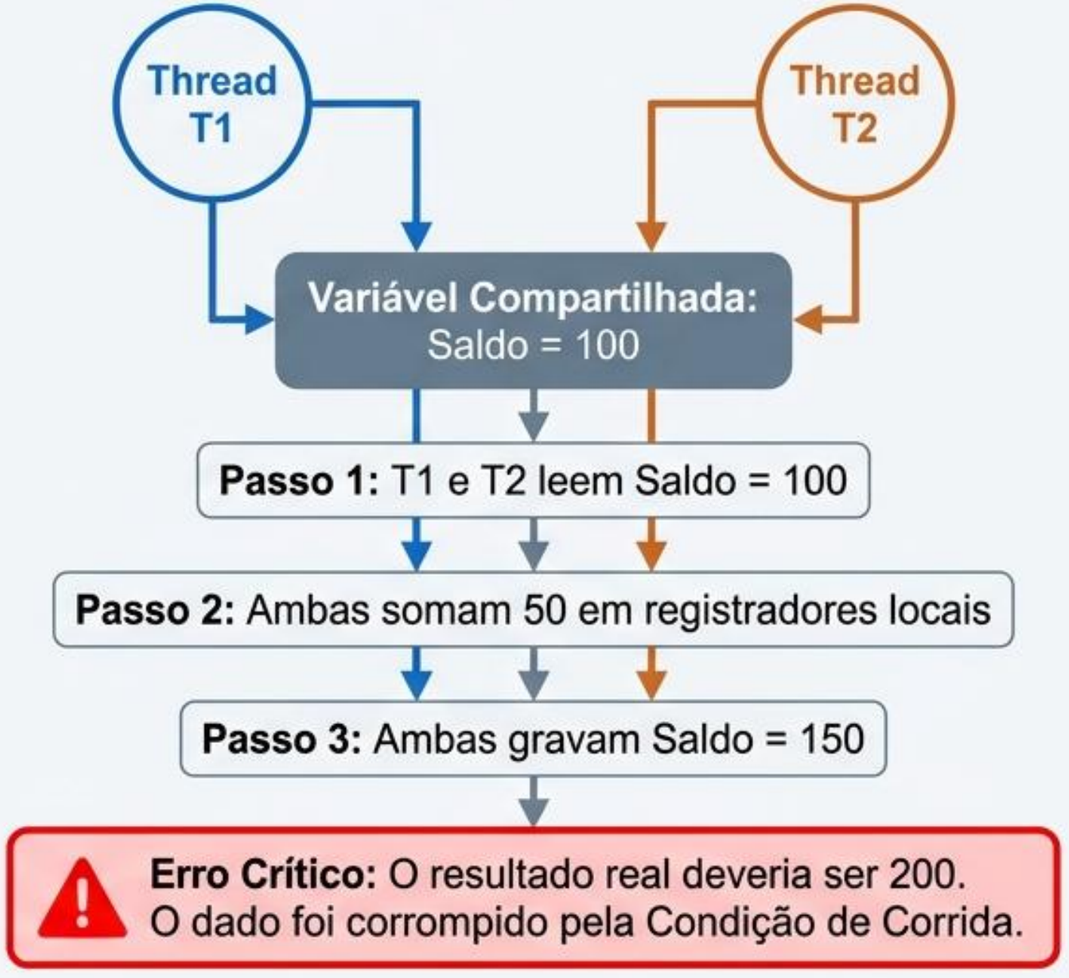
Instituição: UNEMAT – Universidade do Estado de Mato Grosso | Campus de Alto Araguaia

Gemini Notebook

O Caos Físico: Mundo Real
Mundo Rerl



O Caos Computacional: Sistema Operacional



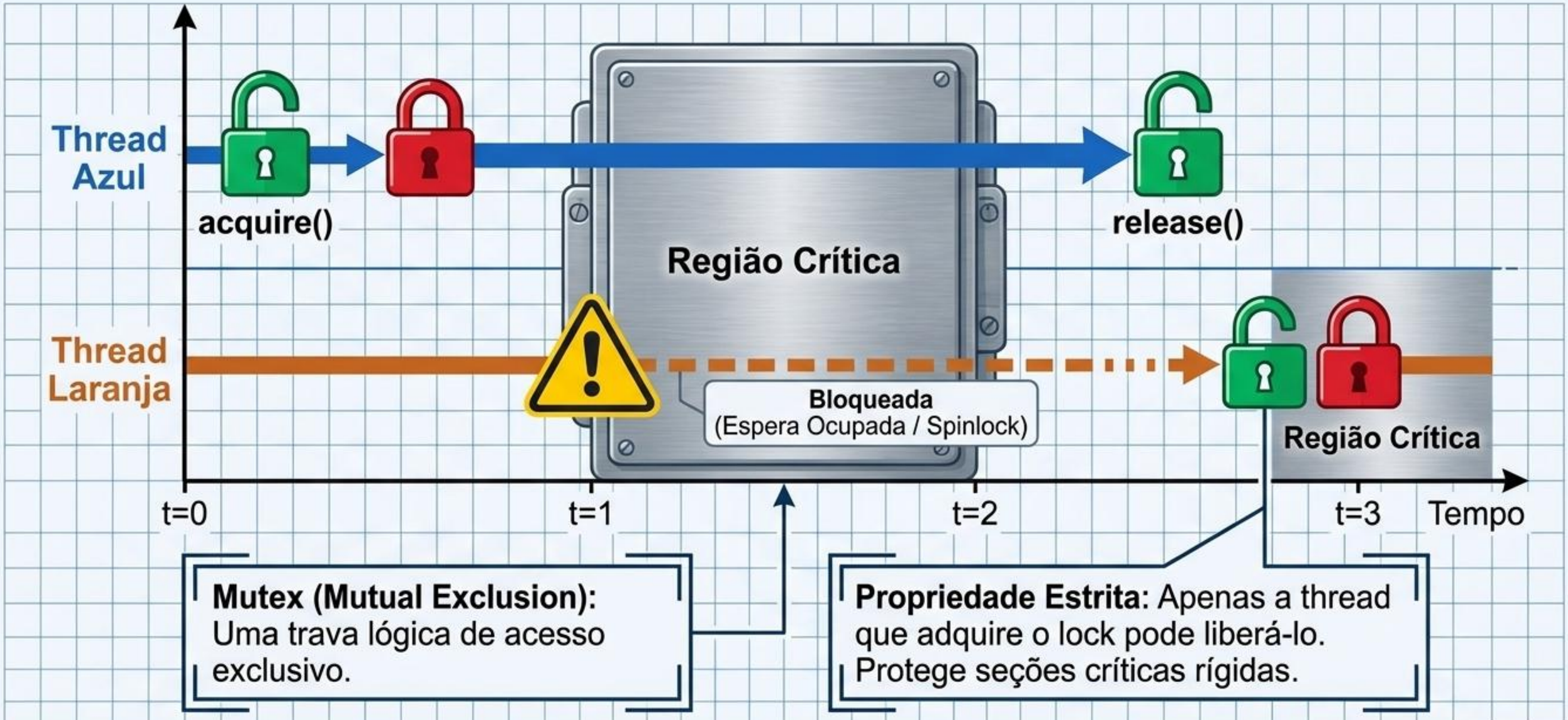
Concorrência: Múltiplas threads ativas simultaneamente.

Região Crítica: O trecho de código onde o recurso compartilhado é acessado.

Exclusão Mútua: O princípio de que apenas UM ator acessa o recurso por vez.

Gemini Notebook

Mutex: A Chave de Acesso Exclusivo



Mutex (Mutual Exclusion): Uma trava lógica de acesso exclusivo.

Propriedade Estrita: Apenas a thread que adquire o lock pode liberá-lo. Protege seções críticas rígidas.

Gemini Notebook

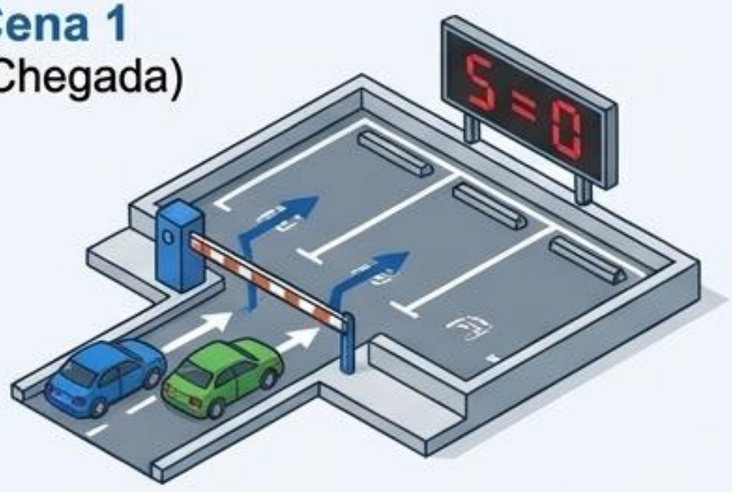
Semáforos: O Controle por Contagem



Mecânica Técnica:

- Baseado em contador inteiro (S) e filas do SO.
- **wait(S)**: Decrementa. Se S < 0, bloqueia.
- **signal(S)**: Incrementa. Acorda thread na fila.
- Operações são obrigatoriamente ATÔMICAS.

Cena 1
(Chegada)



3 threads executam **wait(S)**.
Cancela abre 3 vezes.

Cena 2
(Fila de Espera)



Thread Laranja executa **wait(S)**.
Como S=0, ela vai para a Fila de Espera.

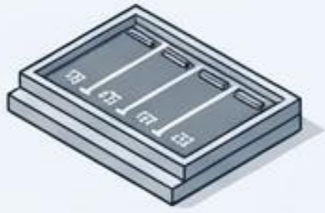
Cena 3
(Retomada)



Carro sai e executa **signal(S)**.
S incrementa provisoriamente.
O sistema acorda a Thread Laranja

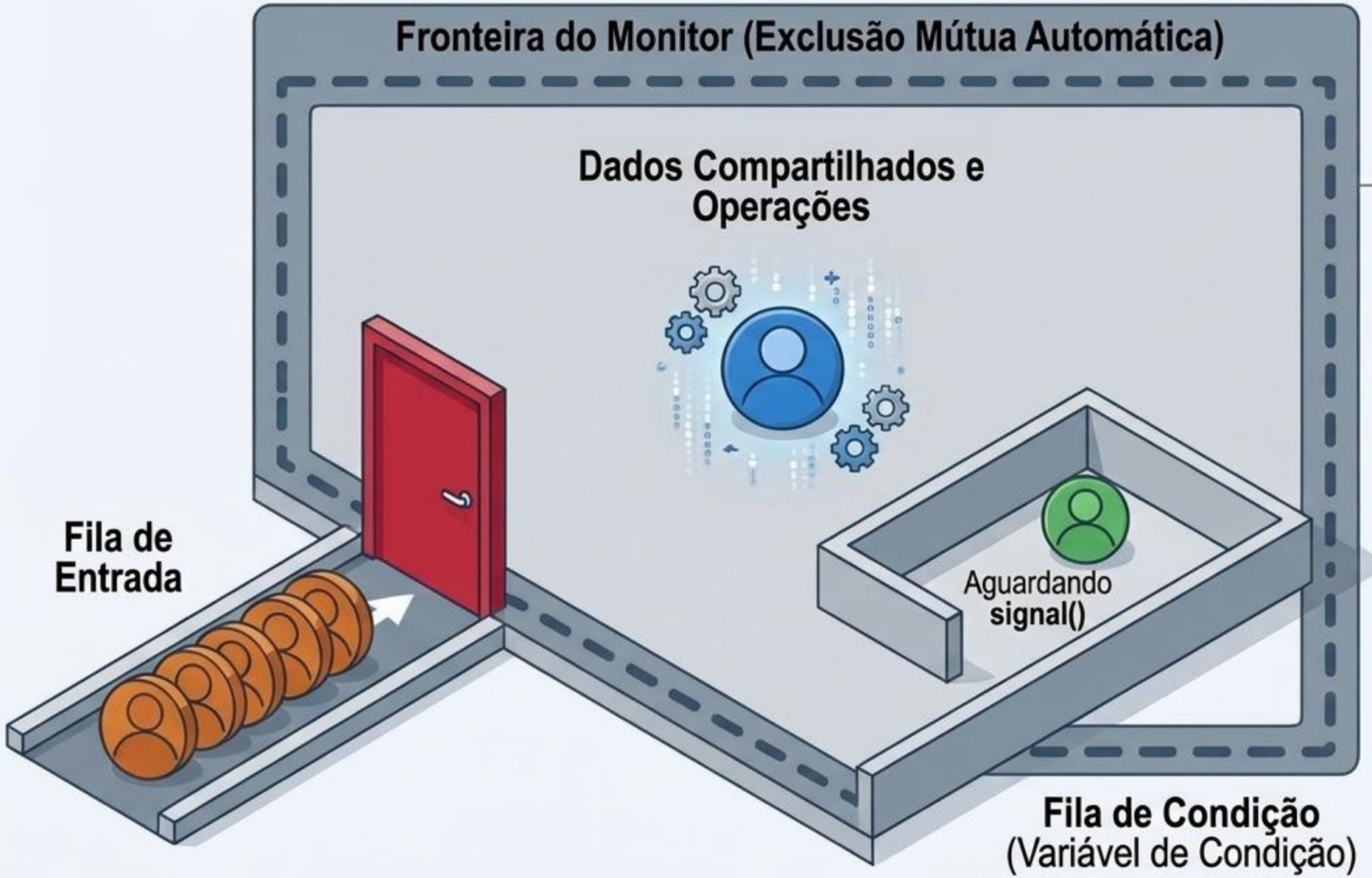
Gemini Notebook

Matriz de Diagnóstico: Mutex vs. Semáforos

Critério	Mutex	Semáforo Binário	Semáforo Contador
			
Limites de Valor	0 ou 1	0 ou 1	0 a N
Propriedade (Ownership)	Sim. Quem trava, obrigatoriamente destrava.	Não. Qualquer thread pode dar signal.	Não.
Tratamento de Fila	Espera Ocupada (Spinlock) ou Bloqueio.	Bloqueio via Escalonador.	Bloqueio via Escalonador.
Melhor Caso de Uso	Proteção estrita de 1 Região Crítica.	Sinalização rápida entre produtor-consumidor.	Gerenciamento de um ‘pool’ de instâncias idênticas (ex: conexões, vagas).

Gemini Notebook

Monitores: A Instalação Gerenciada e Encapsulada



Encapsulamento Nativo

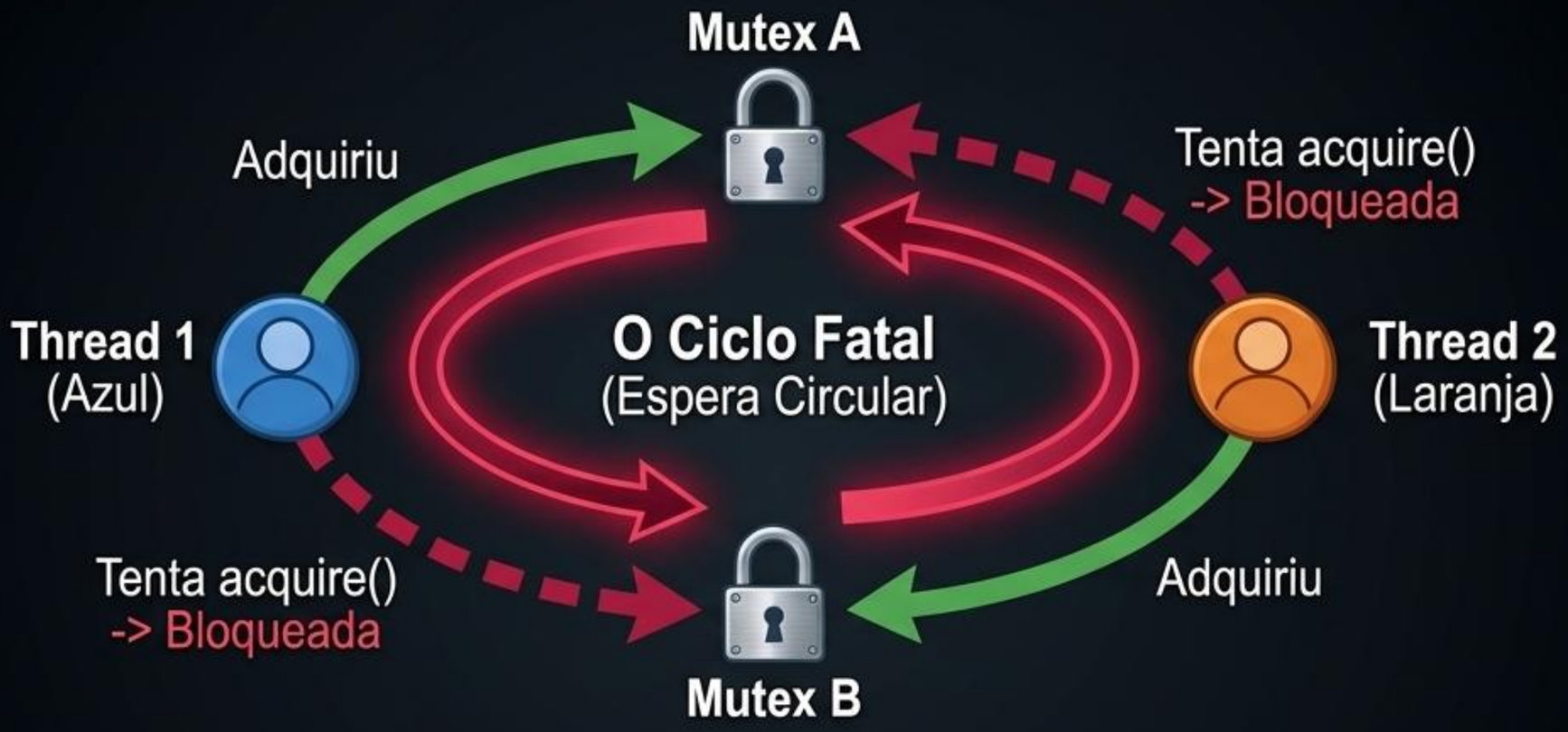
- O compilador (ex: Java “synchronized”) insere os locks automaticamente.
- Apenas uma thread pode estar ativa dentro do monitor.

Variáveis de Condição

- Mecanismo interno: **wait()** permite que a thread saia temporariamente da execução ativa se uma condição não for atendida, liberando o monitor para outras.
- **signal()** acorda quem está nesta fila interna.

Gemini Notebook

O Colapso: Introdução ao Deadlock



Deadlock

Paralisia cíclica e permanente. Ninguém avança.

Starvation

Injustiça do escalonador. Uma thread é eternamente preterida, mas o sistema avança.

Espera Normal

Atraso rotineiro e esperado aguardando a liberação de um recurso.

Gemini Notebook

Anatomia do Deadlock: As 4 Condições de Coffman

1. Exclusão Mútua

Ao menos um recurso em disputa é não-compartilhável (requer acesso exclusivo).



2. Posse e Espera (Hold and Wait)

Um processo retém recursos já alocados enquanto aguarda para obter recursos adicionais presos por outros.

3. Ausência de Preempção

O SO não pode tomar recursos à força; a thread deve liberá-los voluntariamente.

4. Espera Circular

Forma-se um ciclo fechado perfeito de dependências entre processos e recursos.

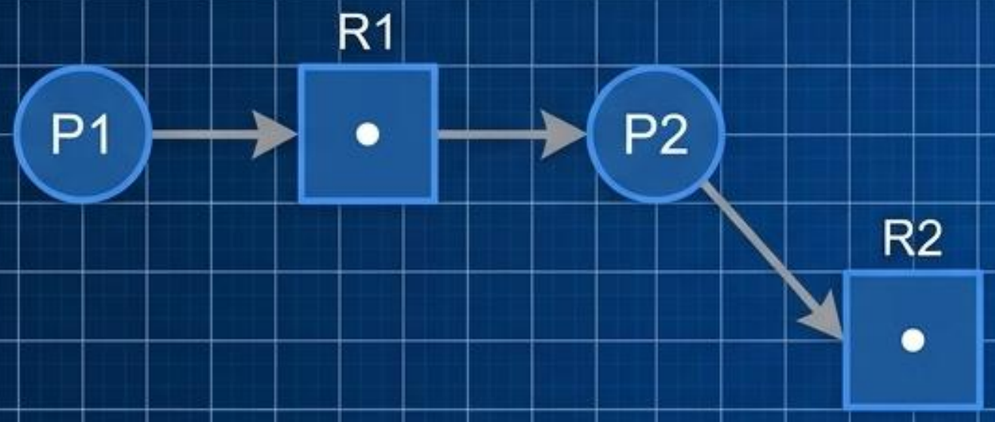
AVISO RIGOROSO: O Deadlock só ocorre se as quatro condições forem verdadeiras de forma SIMULTÂNEA. Remover um único pino destrava a engrenagem.

Gemini Notebook

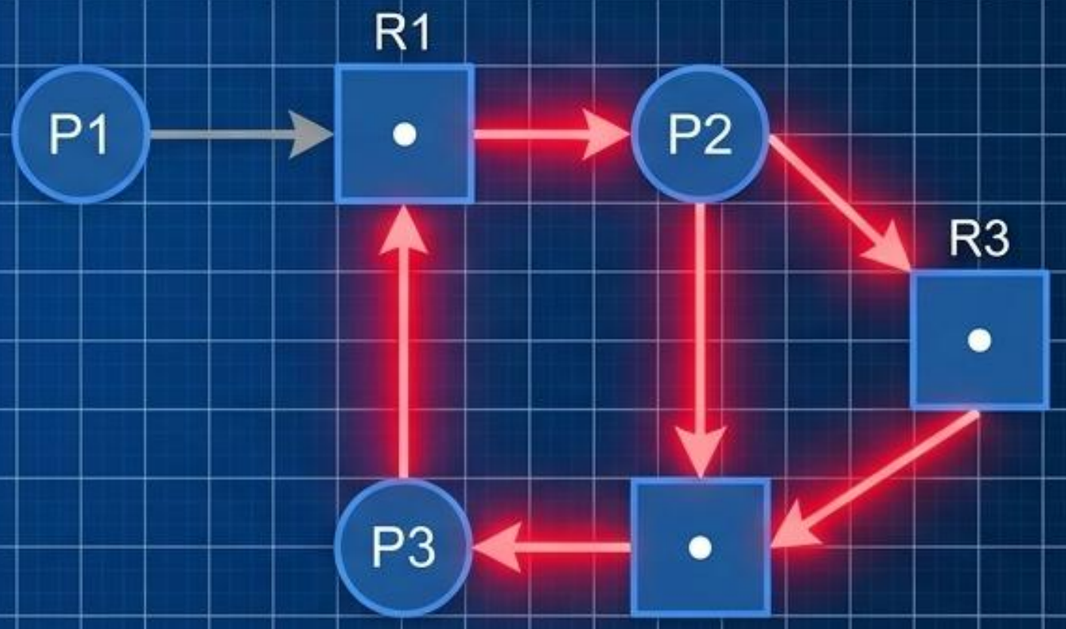
O Grafo de Alocação de Recursos (RAG)

Nós: Círculo = Processo (P) | Quadrado = Recurso (R) | Pontos no quadrado = Instâncias do recurso
Arestas: Seta P -> R = Solicitação | Seta R -> P = Atribuição

Estado Saudável (Sem Ciclo)



Estado Fatal (Deadlock)



Nota Matemática: Se existe ciclo num grafo de instâncias únicas, o Deadlock é garantido. Em grafos de instâncias múltiplas, o ciclo é condição necessária, mas não suficiente.

Gemini Notebook

Tratamento de Deadlock: O Quadrante Estratégico



Curiosidade Arquitetural: A grande maioria dos SOs modernos (Linux, Windows) adota o Algoritmo do Avestruz para otimizar o desempenho geral, cobrando rigor dos desenvolvedores.

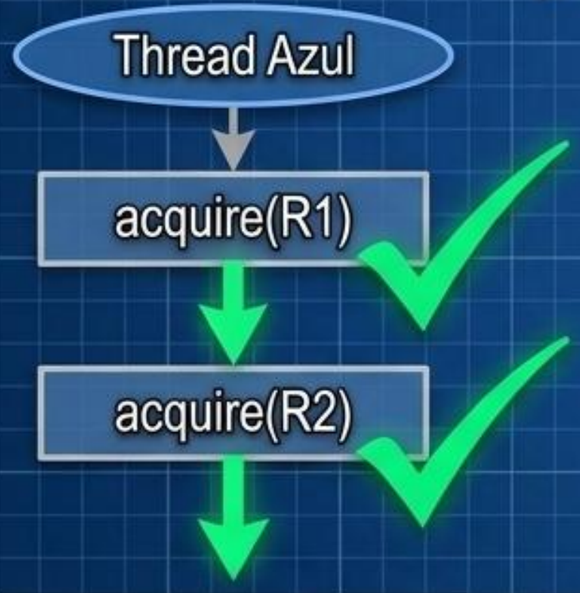
Gemini Notebook

Prevenção: Quebrando a Espera Circular

Régua de Prioridade Global

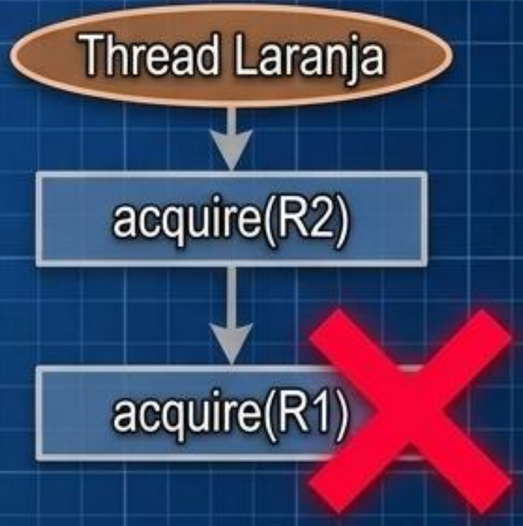
Ordem Obrigatória de Aquisição: R1 -> R2 -> R3

Código Estruturalmente Seguro



A thread respeita a hierarquia global. Execução permitida.

Código Inseguro Interceptado



A thread tenta pedir um recurso de ID menor após reter um maior. O compilador/sistema aborta no design.

Conceito Chave: Ao forçar uma hierarquia global estrita para a aquisição de múltiplos locks, torna-se matematicamente impossível fechar um ciclo. O ***deadlock*** é abortado estruturalmente.

Gemini Notebook

Evitação, Detecção e Recuperação

Evitação



Dinâmica:
Usa o Algoritmo do Banqueiro. O SO simula cada alocação verificando se haverá um "caminho seguro" para finalizar todas as tarefas. Se não, bloqueia o pedido preventivamente.

Limitação:
Overhead matemático extremo. Exige saber o consumo máximo de recursos de cada processo antecipadamente, o que é irrealista hoje.

Detecção



Dinâmica:
O SO possui uma thread em background que varre o Grafo de Alocação de Recursos (RAG) periodicamente, buscando identificar ciclos fechados.

Limitação:
Consome ciclos preciosos de CPU apenas procurando problemas, sem realizar nenhum trabalho útil para evitá-los.

Recuperação

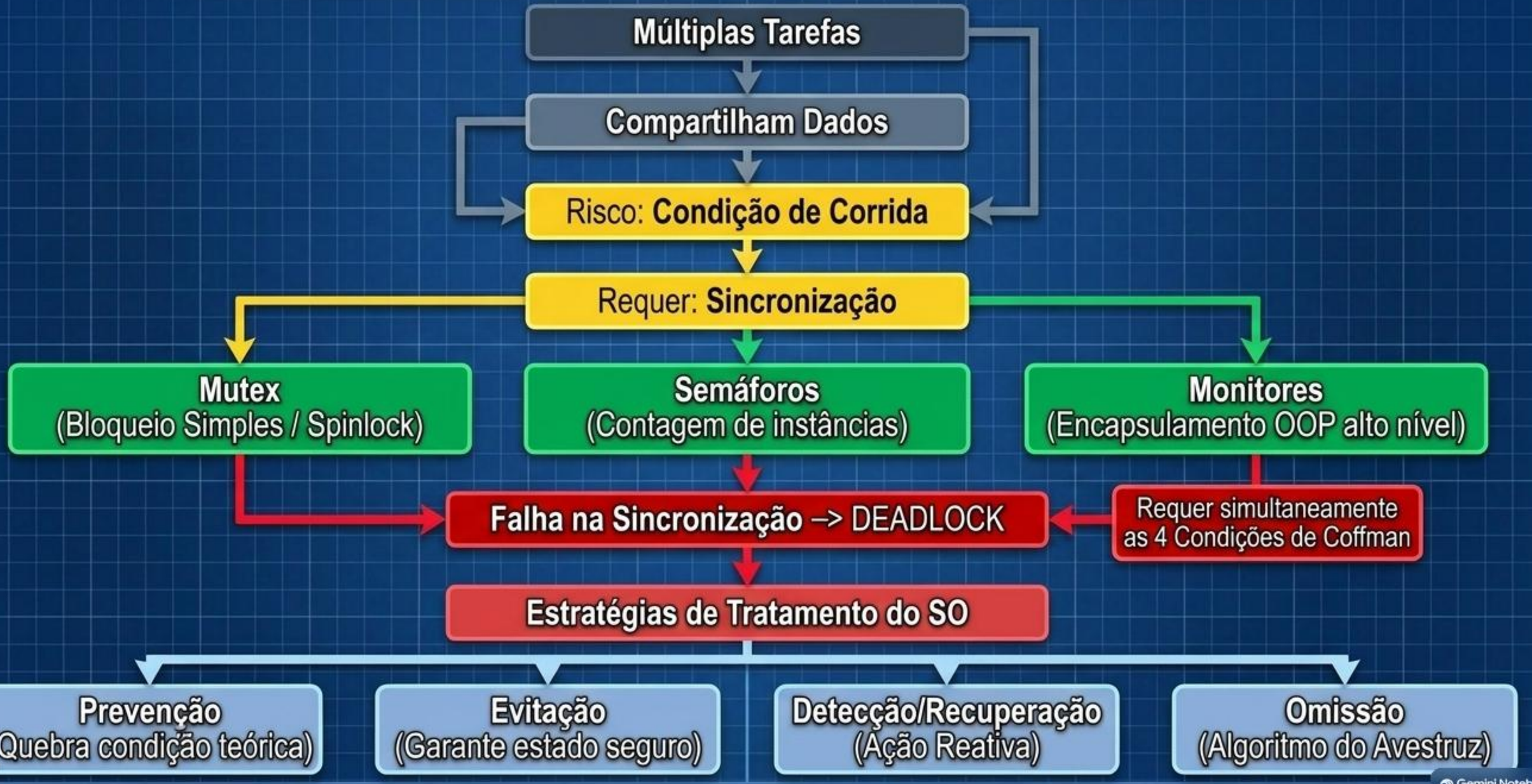


Dinâmica:
Uma vez detectado o deadlock, o SO escolhe um processo "vítima". Ele aborta o processo ou realiza preempção forçada (rouba o lock).

Limitação:
Custo altíssimo de Rollback. Pode gerar perda abrupta de dados em execução ou corromper arquivos abertos.

Gemini Notebook

Síntese Estrutural: O Mapa da Concorrência



Gemini Notebook

Exercício Resolvido: Análise de Risco em Código

```
Thread 1:
1. lock(M1);
2. lock(M2);
3. executa_tarefa();
4. unlock(M2); unlock(M1);

Thread 2:
5. lock(M2);
6. lock(M1);
7. executa_tarefa();
8. unlock(M1); unlock(M2);
```

Pergunta 1: Há deadlock?
Resposta: Sim. Se houver interrupção do escalonador exatamente após a Thread 1 travar M1 e a Thread 2 travar M2.

Pergunta 2: Identifique as Condições de Coffman explícitas.
Resposta: Posse e Espera (T1 tem M1 e pede M2). Espera Circular (T1 quer o que T2 tem; T2 quer o que T1 tem).

Pergunta 3: Qual estratégia de Prevenção consertaria o código?
Resposta: Aplicar ordenação global. Alterar a Thread 2 para requisitar na mesma ordem que a Thread 1 (primeiro M1, depois M2).

Gemini Notebook

Questões Conceituais de Aprofundamento

1. Por que um Lock Mutex costuma ser implementado internamente pelo Kernel através de 'Espera Ocupada' (Spinlocks)?

Porque a troca de contexto pelo escalonador (colocar a thread para dormir e acordar outra) é mais cara e lenta do que o processador gastar alguns ciclos aguardando em loop, desde que a região crítica seja muito curta.

2. Em Variáveis de Condição de um Monitor, por que usar 'signal()' é fundamentalmente diferente de um Semáforo?

Semáforos possuem estado (memória). Um signal num semáforo vazio é guardado para a próxima thread. Num Monitor, se ninguém aguarda na fila, o signal é sumariamente descartado e perdido.

3. Na prática, por que sistemas Linux e Windows abamdonaram o uso de Evitação (Algoritmo do Banqueiro)?

Ambientes modernos são altamente imprevisíveis. É impossível e irrealista exigir que cada aplicativo ou aba de navegador declare todas as suas necessidades máximas de memória e recursos no instante exato de sua inicialização.

Gemini Notebook